

BetterSearch — technical architecture

A semantic search prototype over a library catalogue: 4,114 records, searchable by meaning rather than by the words a reader happens to have typed. It ships in two shapes — a Python service, and a single HTML file that runs the whole thing in a browser with no server behind it.

This document describes what the system is made of. **Part I** is the shape of it, and assumes engineering literacy but no familiarity with this repository. **Part II** is the component-by-component detail, and assumes you are about to change something.

Everything here is named by real path and real symbol, so it can be checked against the code rather than believed. It describes structure only; the measured results that justify particular choices live in `README.md` and in the module docstrings, next to the code they argue for, where they cannot drift out of date unnoticed.

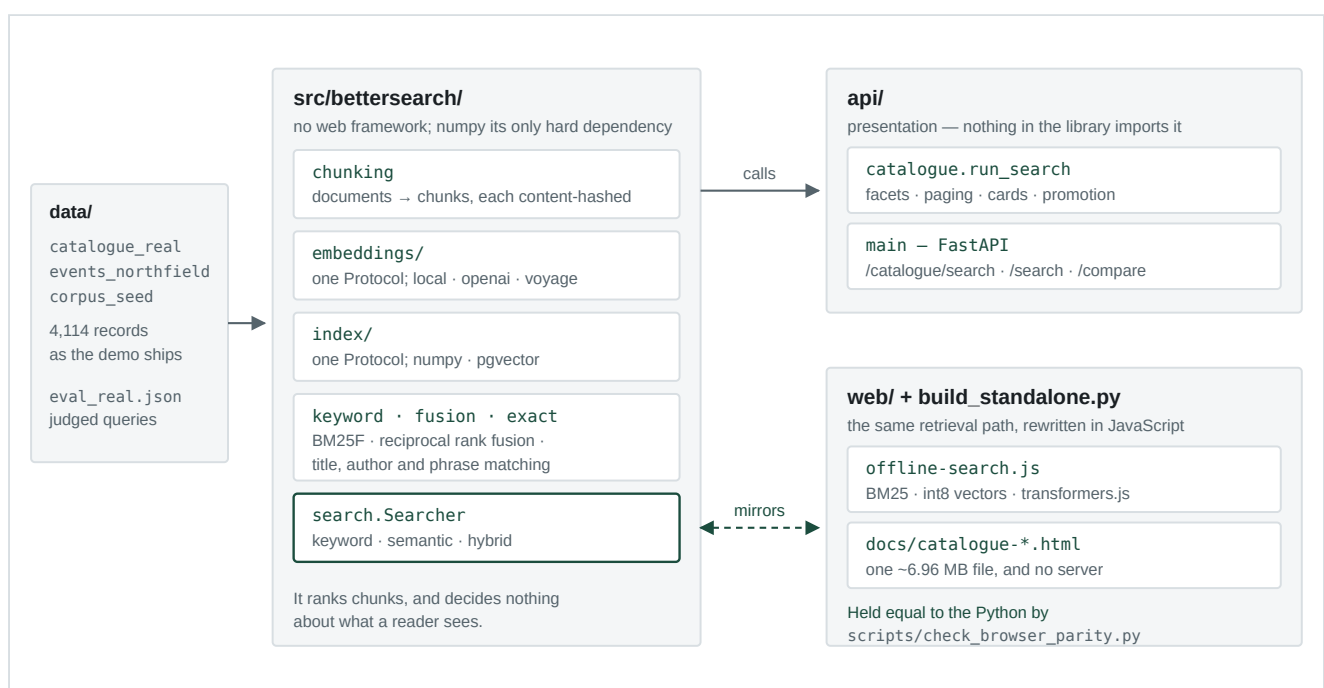
Part I — The high-level view

1. What it does

A reader types a phrase. The system returns catalogue records ordered by how well they answer it, with facets to narrow by, and a line on each card saying why that record is there. It can do this two ways: by matching the words themselves, or by matching meaning — so that *learning to be present* can return a book about attention whose record never uses either word.

Retrieval is the whole product. There is no generation step, no chat, and nothing is written back into a record. Where the system has used a language model to make thin records findable, that text is a bridge for retrieval only and is never shown to a reader as though it were catalogue truth.

2. The shape: three trees



TREE	WHAT IT IS	WHAT IT MAY IMPORT
<code>src/bettersearch/</code>	the library: chunking, embedding, indexing, ranking	nothing above it
<code>api/</code>	presentation — turns a ranking into a page of cards	the library
<code>web/</code>	the same retrieval path, written again in JavaScript	nothing; it is a port
<code>scripts/</code>	build, measure, tune, render	anything

The dependency arrows only point one way. The library has no idea a web framework exists; `numpy` is its only hard dependency, and everything else — `sentence-transformers`, `fastapi`, `anthropic`, `psycopg` — is an optional extra installed per deployment. Nothing in `src/bettersearch/` imports `api/`, which is what makes the library droppable into a Django or DRF project without carrying a second web framework in with it.

`web/` is the unusual one. It is not a client that calls the API; it is a **re-implementation** of the retrieval path in JavaScript, so the same search can run with no server at all. Two implementations of one algorithm is a liability, and it is managed rather than hoped about: `scripts/check_browser_parity.py` runs both sides over the same queries and reports where they disagree.

3. Two deployment shapes, one source

The server. `uvicorn api.main:app` — FastAPI over the library, with the index either as local `numpy` arrays or as a `pgvector` table in Postgres. This is the shape that scales, and the one a real integration would use.

The standalone file. `scripts/build_standalone.py` inlines the corpus, its vectors and the entire search engine into one ~6.96 MB HTML file. Opening it runs a real semantic search with no server, no API key and no network after the model has been fetched once. This is the shape you can email to someone.

Both run the same encoder. The browser stores its corpus vectors as `int8` rather than `float32`, which is the one remaining difference and is measured rather than assumed. The page shipped a smaller model until September 2026, on an argument about download size that turned out to be wrong about the download size; the cost was a demo that behaved differently from the service it was demonstrating.

4. The four ideas the design rests on

Chunk, then embed. A document is split into overlapping passages before embedding, because an embedding of a whole document is an average, and averages retrieve badly. Each chunk carries its parent `doc_id`, so results can be collapsed back to one row per record at the point of display.

The index is the contract. Every index stores the `model_id` it was built with, and that id is checked on read *and* on write. Change the encoder and the index refuses to answer rather than quietly scoring today's query against yesterday's vectors — the failure mode that produces plausible, wrong results and no error. `scripts/build_standalone.py` enforces the same rule at build time, refusing to bake a page whose model disagrees with its index.

Retrieval and presentation are separate. `Searcher` ranks chunks and decides nothing about what a reader sees. Everything that makes a page — collapsing chunks to records, folding an event series into one card, counting facets, paging, deciding which explanation to attach — lives in `api/catalogue.py`. The

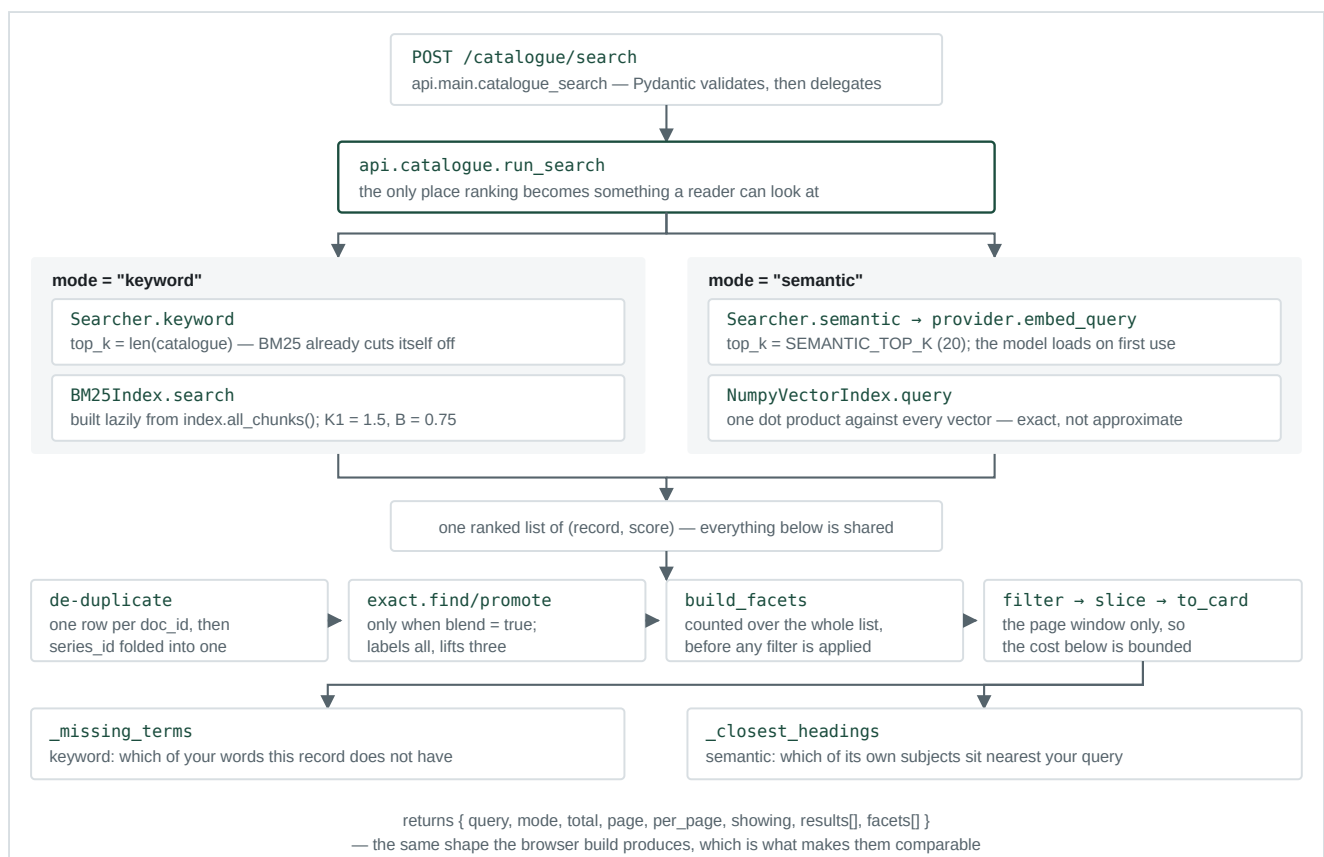
split matters because presentation changes constantly and retrieval should not have to move each time it does.

The browser port is held true by a check, not by discipline. Two implementations drift.

`check_browser_parity.py` drives the built page in a real headless browser, runs the same queries through the Python API, and compares rankings, explanations and promotion labels record by record. Exact agreement is the bar for the model-free parts; the parts that go through a quantised model are expected to differ slightly, and the check reports how much rather than pretending they don't.

Part II — The low-level view

5. A request, end to end



`POST /catalogue/search` → `api.main.catalogue_search` (Pydantic validates the body) → `api.catalogue.run_search`, which does the following in order:

1. **Rank.** `mode="semantic"` → `Searcher.semantic(top_k=SEMANTIC_TOP_K)`, which embeds the query and hands it to `NumpyVectorIndex.query` — one dot product against every vector, exact rather than approximate. `mode="keyword"` → `Searcher.keyword(top_k=len(catalogue))` at full depth, because BM25 cuts itself off by returning only positive scores.
2. **Collapse to records.** De-duplicate by `doc_id`, drop hits no longer in the catalogue, and fold repeated `series_id` occurrences into their best-ranked instance, incrementing a `repeats` count.
3. **Promote** (only when `blend=True`): `exact.find` locates records the reader arguably *named*; `exact.promote` labels every one of them and lifts the first `MAX_PROMOTED` to the top.

4. **Facet.** `build_facets` counts over the whole ranked list *before* any filter is applied, so the counts describe what is available rather than what survived the last click.
5. **Filter and page.** `matches_filters` (OR within a facet, AND across facets), then slice the page window, then `to_card` each surviving record.
6. **Explain.** Keyword mode attaches `missing_terms` — which of your words this record does not contain. Semantic mode attaches `closest_headings` — which of the record's own subject headings sit nearest your query. Both run on the page window only, so their cost is bounded by `per_page` rather than by the corpus.

The response is `{query, mode, total, page, per_page, showing, results[], facets[]}`. The browser build produces the identical shape, which is what makes the two comparable at all.

Hybrid mode (`reciprocal_rank_fusion` over both lanes, each retrieved at four times the requested depth) exists on `/search` and `/compare` but is deliberately **not** reachable from the catalogue page.

6. Component summary

The library — `src/bettersearch/`

MODULE	WHAT IT DOES	NAMES AND VALUES THAT MATTER
<code>types.py</code>	the shared vocabulary; no behaviour	<code>Document</code> , <code>Chunk</code> , <code>EmbeddedChunk</code> , <code>ScoredChunk</code> , <code>IndexStats</code> , and the two fatal errors <code>ModelMismatchError</code> , <code>EmptyIndexError</code>
<code>config.py</code>	one frozen <code>Settings</code> , built entirely from the environment	<code>load_settings()</code> ; every knob is a <code>BETTERSEARCH_*</code> variable (§9)
<code>chunking.py</code>	paragraph-aware splitting with overlap	<code>chunk_document</code> , <code>chunk_documents</code> , <code>content_hash</code> (sha256, first 16 hex); ~450-token target, 60-token overlap, tokens estimated at 1.3 per word; oversized paragraphs split on sentences
<code>keyword.py</code>	the BM25F baseline, hand-rolled	<code>tokenize</code> (56-word stopwords list), <code>BM25Index</code> ; <code>K1 = 1.5</code> , <code>B = 0.75</code> , <code>TITLE_WEIGHT = 1.0</code> . Returns only positively-scoring documents — a query sharing no terms returns nothing at all
<code>fusion.py</code>	reciprocal rank fusion	<code>reciprocal_rank_fusion</code> , <code>DEFAULT_K = 60</code> . Uses list position only, never a raw score, so two incomparable scales can be combined
<code>exact.py</code>	the one signal neither ranking lane can see: word order	<code>find</code> , <code>promote</code> , <code>ExactMatch</code> ; <code>PHRASE_TOKENS = 3</code> , <code>MAX_PROMOTED = 3</code> . Pure string work over title and author — no model, no vectors. Labelling and lifting are separate jobs; the cap governs only the second
<code>search.py</code>	the public entry point	<code>Searcher.keyword</code> / <code>.semantic</code> / <code>.hybrid</code> / <code>.search</code> / <code>.compare</code> / <code>.stats</code> ; <code>MODES = ("keyword", "semantic", "hybrid")</code> . Each hybrid arm retrieves 4× the requested depth before fusing
<code>ingest.py</code>	documents → chunks → vectors → index	<code>load_corpus</code> (merges several files; a repeated <code>doc_id</code> raises), <code>ingest_documents</code> , <code>ingest_corpus</code> , <code>IngestReport</code> ; embeds in batches of 64
<code>evaluate.py</code>	retrieval metrics at document level	<code>recall_at_k</code> , <code>reciprocal_rank</code> , <code>ndcg_at_k</code> , <code>evaluate_mode</code> , <code>evaluate_all</code> , <code>per_query_breakdown</code>
<code>cli.py</code>	<code>bettersearch</code> on the command line	<code>ingest</code> , <code>search</code> , <code>compare</code> , <code>evaluate</code> , <code>enrich</code> , <code>stats</code>

Embeddings — `src/bettersearch/embeddings/`

One `Protocol`, three implementations. Anything with `model_id`, `dimensions`, `max_input_tokens`, `embed_documents` and `embed_query` is a provider.

MODULE	PROVIDER	NOTES
<code>base.py</code>	the <code>EmbeddingProvider</code> Protocol	plus <code>l2_normalize</code> , so a dot product <i>is</i> cosine similarity everywhere downstream
<code>local.py</code>	sentence-transformers, the default	BAAI/bge-base-en-v1.5 (768d). <code>SUGGESTED_MODELS</code> carries seven encoders with their input limits, dimensions and download sizes
<code>openai.py</code>	<code>text-embedding-3-small</code>	8,191-token limit; needs <code>OPENAI_API_KEY</code> ; re-sorts the response by index because the API does not promise order
<code>voyage.py</code>	<code>voyage-3.5-lite</code>	32,000-token limit; asymmetric — documents and queries are encoded with different <code>input_type</code>

Both hosted providers support Matryoshka truncation, which is why `embedding_dimensions` is a setting rather than a property of the model.

Index — `src/bettersearch/index/`

One Protocol — `upsert`, `query`, `existing_hashes`, `all_chunks`, `stats`, `clear` — and two backends.

MODULE	BACKEND	STORAGE
<code>numpy_index.py</code>	<code>numpy</code> , the default	two files: <code><path>.npz</code> (float32 vectors) and <code><path>.json</code> (chunks, <code>model_id</code> , <code>dimensions</code>). Brute-force exact cosine over every row
<code>pgvector_index.py</code>	<code>pgvector</code>	one table <code>bettersearch_chunk</code> : a <code>halfvec</code> embedding column with an HNSW cosine index, plus a generated <code>tsvector</code> with a GIN index for full-text

Both raise `ModelMismatchError` rather than answer with the wrong encoder's vectors, and `EmptyIndexError` rather than return an empty list that looks like "no results".

Enrichment — `src/bettersearch/enrichment/`

Thin catalogue records — a title, an author, a couple of subject headings — give an encoder almost nothing to work with. This subsystem asks a language model for bridging vocabulary and stores it beside the record. **The generated text is used only to make a record findable. It is never displayed as catalogue truth.**

MODULE	WHAT IT DOES
<code>schema.py</code>	<code>Enrichment</code> , the strict JSON schema the model must return, <code>PROMPT_VERSION = "v1"</code> , and <code>source_hash</code>
<code>prompt.py</code>	the system prompt, which branches on whether the model recognises the work — a known book is described from knowledge, an unknown one only from its own record
<code>client.py</code>	the Anthropic client: single calls, batch submission, and a <code>preflight</code> that proves one item works before paying for a batch
<code>store.py</code>	append-only JSONL, keyed on <code>(item_id, prompt_version, model, source_hash)</code> — so nothing is ever paid for twice and a changed record re-enriches by itself
<code>pipeline.py</code>	resumable orchestration; an in-flight batch id is written to a sidecar so a killed process reattaches instead of losing paid work

Presentation — `api/`

MODULE	WHAT IT DOES
<code>main.py</code>	the FastAPI app. Holds no retrieval logic: seven routes, a <code>{status, data}</code> envelope, and two exception handlers that turn <code>EmptyIndexError</code> and <code>ModelMismatchError</code> into HTTP 409 rather than a 500
<code>catalogue.py</code>	everything that makes a page. <code>run_search</code> (§5), <code>build_facets</code> , <code>matches_filters</code> , <code>to_card</code> , <code>_missing_terms</code> , <code>_closest_headings</code> , <code>load_catalogue</code> , <code>load_meta</code> . <code>SEMANTIC_TOP_K = 20</code> , <code>EXPLAIN_HEADINGS = 2</code>

`_closest_headings` embeds each distinct subject heading on the current page and caches the vectors in a `WeakKeyDictionary` **keyed on the provider object**, so the cache dies with the provider it belongs to and cannot outlive a model swap.

Browser — `web/`

FILE	WHAT IT DOES
<code>offline-search.js</code>	the port. <code>tokenize</code> , <code>buildBM25</code> , <code>keywordRank</code> , <code>semanticRank</code> , <code>missingTerms</code> , <code>closestHeadings</code> , <code>findExact</code> , <code>promoteExact</code> , <code>collapse</code> , <code>buildFacets</code> , <code>search</code> . Exposes one global, <code>window.OFFLINE</code>
<code>catalogue.html</code>	the catalogue page. Talks to <code>/catalogue/search</code> when served, and to <code>window.OFFLINE</code> when that global exists — the same page source is both the served app and the standalone file
<code>index.html</code>	a developer comparison view: all three modes side by side, scored. For working on retrieval, not for showing anyone

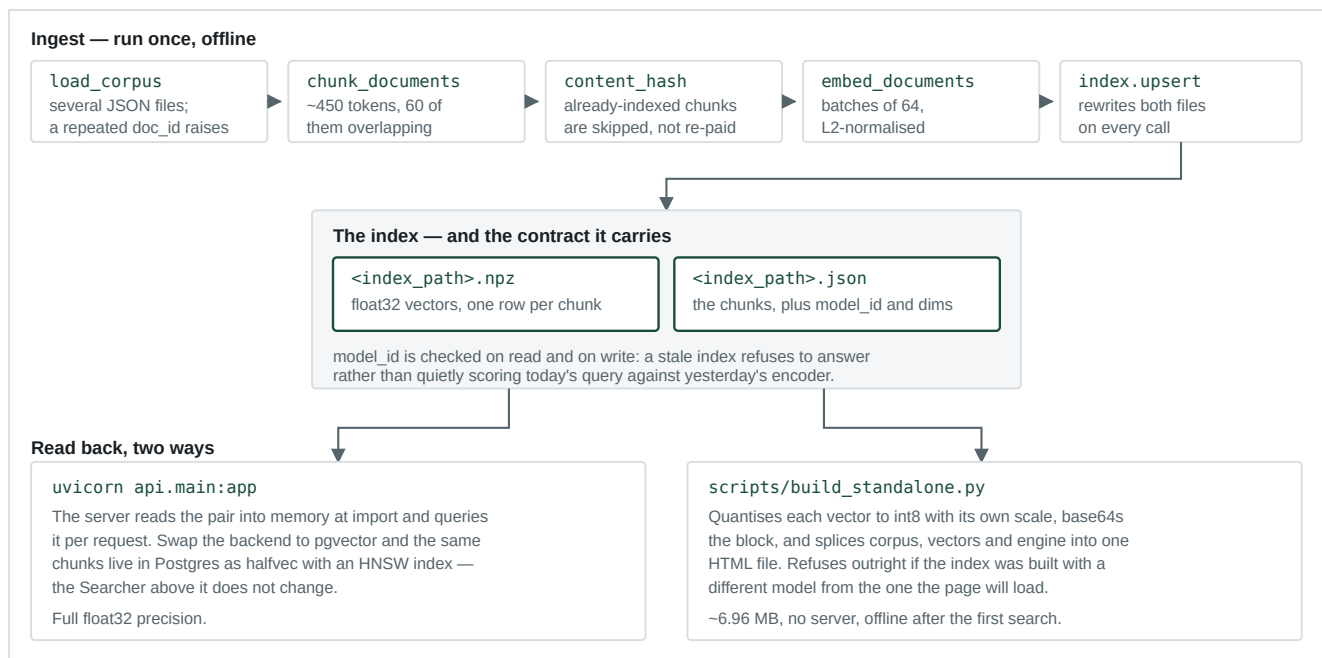
The page loads `@huggingface/transformers` from a CDN on the **first semantic search**, not at page load, and embeds the query with CLS pooling and L2 normalisation to match how the server-side model was trained. Corpus vectors are not re-embedded in the browser; they are lifted from the index at build time, quantised to int8 with a per-vector scale, and base64'd into the file.

Scripts — `scripts/`

Eighteen standalone programs, none imported by anything.

GROUP	SCRIPTS	PURPOSE
Fetch and build data	<code>fetch_openlibrary</code> , <code>fetch_wikipedia</code> , <code>build_catalogue</code> , <code>build_events</code> , <code>synthesise_catalogue</code> , <code>build_eval_set</code>	assemble the corpora and the judged query set
Build artefacts	<code>build_standalone</code>	inline corpus, vectors and engine into one HTML file
Check	<code>check_corpus</code> , <code>check_events</code> , <code>check_browser_parity</code> , <code>check_answerability</code> , <code>check_query_routing</code>	assert properties that tests cannot — corpus coverage, browser/Python agreement, whether a signal exists at all
Tune	<code>tune_title_weight</code> , <code>tune_cutoff</code> , <code>tune_confidence_break</code> , <code>compare_arms</code>	sweeps that chose the constants above, and in several cases argued against shipping a change
Present	<code>render_pdf</code> , <code>capture_screenshots</code>	this PDF, and the README screenshots

7. Data and persistence



Corpora (`data/` , read-only inputs, all committed). Every corpus is `{name, description, presets, documents[]}`. The shipped demo is `catalogue_real.json` (4,000 real bibliographic records from Open Library) plus `events_northfield.json` (114 invented events), sharing one record schema so both collections can be searched and faceted together.

The bibliographic data is real. Availability, copies, branch, format and cover colour are invented, and the events programme is invented entirely. None of it should be quoted as describing a real library.

The index is a file pair, rewritten in full on every `upsert`: `<path>.npz` holds L2-normalised float32 vectors, one row per chunk; `<path>.json` holds the chunks themselves plus the `model_id` and dimensions that make up the contract. The BM25 index is not persisted — it is rebuilt from `all_chunks()` on first keyword search, lazily, so a semantic-only process never pays for it.

Judgements (`eval_real.json` , `eval_real_judgements.json`) are TREC-style pooled relevance labels, kept so that any future change can be measured against the same bar rather than against a new one.

Enrichment is append-only JSONL with an in-flight sidecar, as above.

`.bettersearch/` is gitignored. The index is built at deploy time, not committed — which is why the CI workflow embeds the corpus itself before building a page.

8. Interfaces and contracts

The two Protocols are the extension points. Implement `EmbeddingProvider` to add an encoder; implement `VectorIndex` to add a backend. Neither requires touching `Searcher`.

HTTP.

METHOD	PATH	BODY	RETURNS
GET	/health	—	provider, modes, index stats
POST	/search	query, mode, top_k	scored chunks
POST	/compare	query, top_k	all three modes at once
POST	/catalogue/search	query, mode, page, per_page, filters, blend	cards, facets, paging
GET	/catalogue/meta	—	presets and description, read from the corpus
GET	/catalogue, /	—	the two HTML pages

Note that `/catalogue/search` accepts only `keyword` and `semantic` — `hybrid` is reachable from the developer routes only.

The baked globals. `build_standalone.py` writes five values into the page so the browser runs the same constants as the server rather than a second copy that can drift:

```

window.__SEMANTIC_TOP_K__    window.__EXPLAIN_HEADINGS__    window.__PHRASE_TOKENS__
window.__MAX_PROMOTED__     window.__INTERACTION__

```

`__INTERACTION__` is the only one that changes behaviour rather than a threshold: `"toggle"` keeps the keyword/meaning switch, `"blended"` removes it and routes every search through meaning with named records lifted. It is why two pages are built from one source.

9. Configuration

Everything is an environment variable; there is no config file.

VARIABLE	DEFAULT	EFFECT
BETTERSEARCH_EMBEDDING_PROVIDER	local	local · openai · voyage
BETTERSEARCH_INDEX	numpy	numpy · pgvector
BETTERSEARCH_INDEX_PATH	.bettersearch/index	where the file pair lives
BETTERSEARCH_DATABASE_URL	—	falls back to DATABASE_URL
BETTERSEARCH_CHUNK_TOKENS	450	target chunk size
BETTERSEARCH_CHUNK_OVERLAP	60	overlap between chunks
BETTERSEARCH_EMBEDDING_DIMENSIONS	512	Matryoshka width, hosted providers only
BETTERSEARCH_LOCAL_MODEL	BAAI/bge-base-en-v1.5	the local encoder
BETTERSEARCH_LOCAL_TRUST_REMOTE_CODE	false	required by some long-context encoders; runs code from the model repo, so it is off by default
BETTERSEARCH_CATALOGUE	data/catalogue_library.json	comma-separated corpora to serve
BETTERSEARCH_TOP_K	20	semantic depth for the catalogue page

OPENAI_API_KEY, VOYAGE_API_KEY and ANTHROPIC_API_KEY are read only by the component that needs them. **Search needs no key at all** — only enrichment and the hosted embedding providers do.

10. Tests and CI

pytest -q, five test files and a conftest.py, and the suite needs no model, no network and no API key: conftest.py provides a FakeEmbeddingProvider that derives deterministic vectors from a SHA-256 hash. That is a deliberate limit — **these tests prove the plumbing, not the retrieval quality**. Quality is measured separately, by the scripts in §6, against judged queries.

FILE	COVERS
test_chunking.py	splitting, overlap termination, hash stability, ordinal uniqueness
test_index_and_ingest.py	cosine ordering, the model-mismatch refusal on both read and write, upsert-not-duplicate, incremental re-embedding, colliding doc_ids
test_retrieval.py	tokenising, BM25 behaviour, fusion, the three metrics, mode dispatch, and that keyword mode never loads the embedding model
test_catalogue_api.py	the largest file — paging, facets, filters, series collapsing, card fidelity, explanations, and exact-match promotion
test_enrichment.py	caching, prompt-version invalidation, out-of-order batch matching, and that a dead poller does not lose a paid batch

CI (.github/workflows/ci.yml) installs the dev extra and runs the suite on every push and every pull request, then greps render.yaml to assert the deploy blueprint still builds the index and still installs CPU-only torch.

Pages (`.github/workflows/pages.yml`) embeds the corpus on the runner, builds both standalone pages from that one index, and deploys `docs/` . Because it re-embeds rather than committing vectors, the published files differ from the committed ones by a handful of int8 values — floating-point rounding either side of a quantisation boundary, not a different model.

One gap worth stating plainly: there is no JavaScript test suite. `offline-search.js` is verified only by `check_browser_parity.py` , and that script is not wired into either workflow — it is run by hand. A change to the browser port can therefore reach `main` without anything having compared it to the Python.

Appendix — file map

<code>src/bettersearch/</code>	the library – framework-free, numpy its only hard dependency
<code>types.py</code>	Document, Chunk, ScoredChunk, the two fatal errors
<code>config.py</code>	Settings, from the environment
<code>chunking.py</code>	paragraph splitting, overlap, content hashing
<code>keyword.py</code>	BM25F
<code>fusion.py</code>	reciprocal rank fusion
<code>exact.py</code>	title / author / phrase matching and promotion
<code>search.py</code>	Searcher – the public entry point
<code>ingest.py</code>	documents -> chunks -> vectors -> index
<code>evaluate.py</code>	Recall@5, MRR@10, nDCG@10
<code>cli.py</code>	the bettersearch command
<code>embeddings/</code>	EmbeddingProvider + local / openai / voyage
<code>index/</code>	VectorIndex + numpy / pgvector
<code>enrichment/</code>	schema, prompt, client, store, pipeline
<code>api/</code>	
<code>main.py</code>	FastAPI – seven routes, no retrieval logic
<code>catalogue.py</code>	facets, paging, cards, promotion, explanations
<code>web/</code>	
<code>offline-search.js</code>	the retrieval path, ported to JavaScript
<code>catalogue.html</code>	the catalogue page, served or standalone
<code>index.html</code>	developer comparison view
<code>scripts/</code>	<code>fetch · build · check · tune · present</code>
<code>data/</code>	corpora, judged queries, enrichment JSONL
<code>docs/</code>	the published site, the PDFs, these diagrams
<code>tests/</code>	pytest; fake providers, no network, no keys